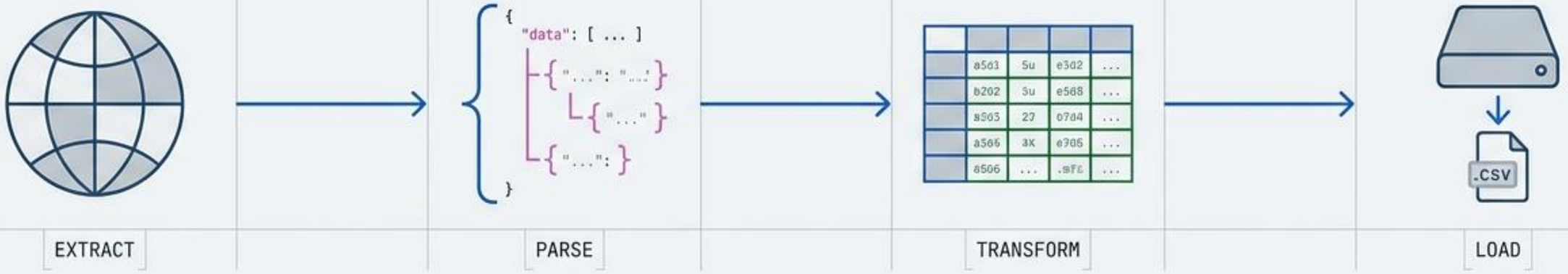
Librerías para Datos y Automatización

De la API al DataFrame: Construyendo un Pipeline de Datos en Python

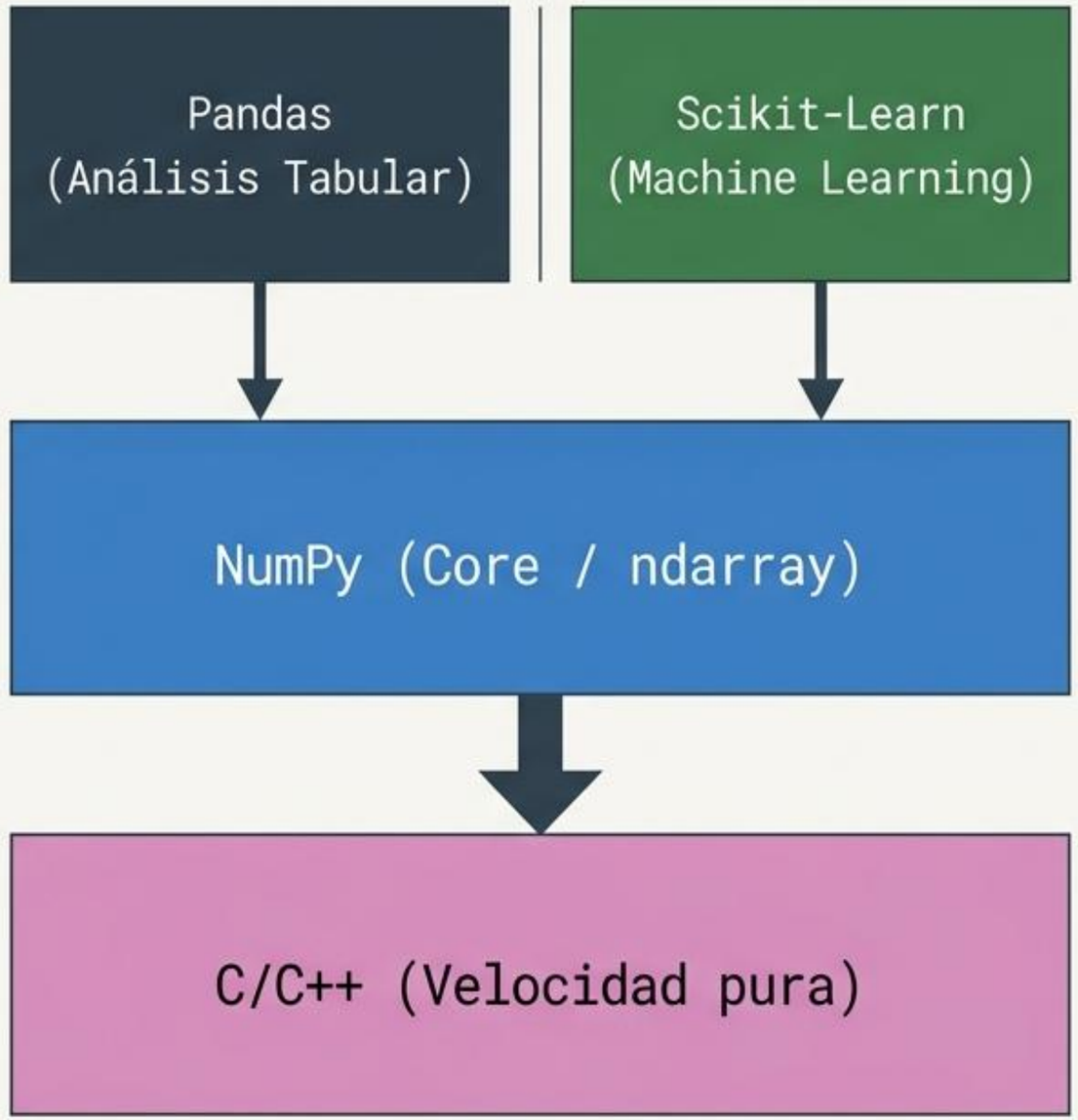


NumPy: El Cimiento de la Ciencia de Datos en Python

Paquete fundamental para computación científica.

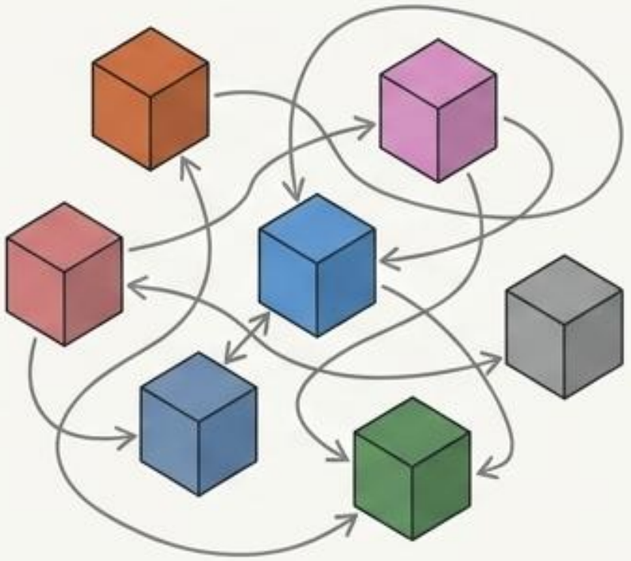
Introduce el objeto ndarray (arreglo multidimensional).

Base sobre la cual operan Pandas y Scikit-Learn.



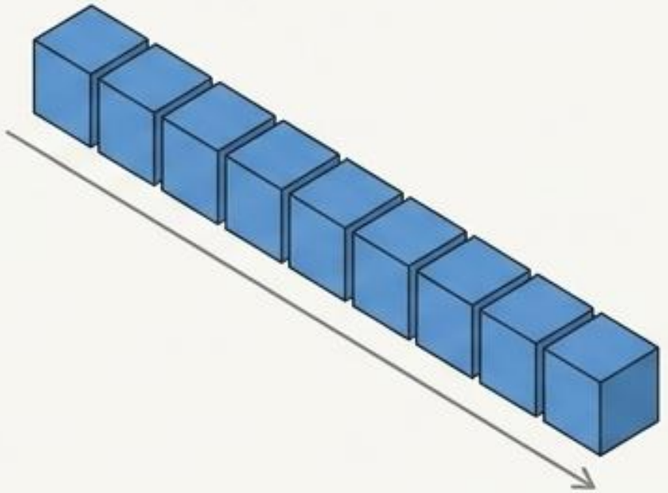
Arrays vs. Listas: La Ventaja de la Memoria Contigua

Lista de Python



Tipos mixtos, memoria dispersa, iteración lenta.

Array de NumPy



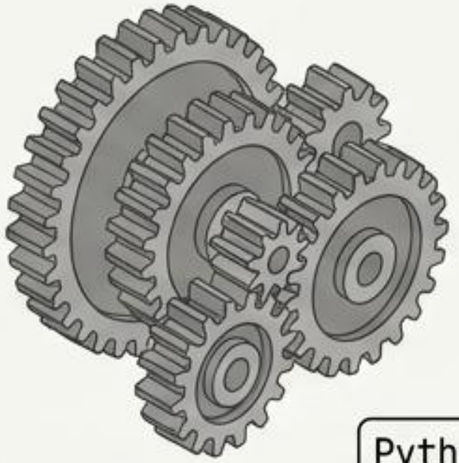
Tipos homogéneos, memoria contigua, acceso instantáneo.

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
ceros = np.zeros((2, 3))
rango = np.arange(0, 10, 2)
```

Vectorización: El Adiós Definitivo a los Bucles Lentos

Bucle For



Python puro (Lento)

```
for i in range(len(array1)):
    array1[i] += 10
```

Vectorización



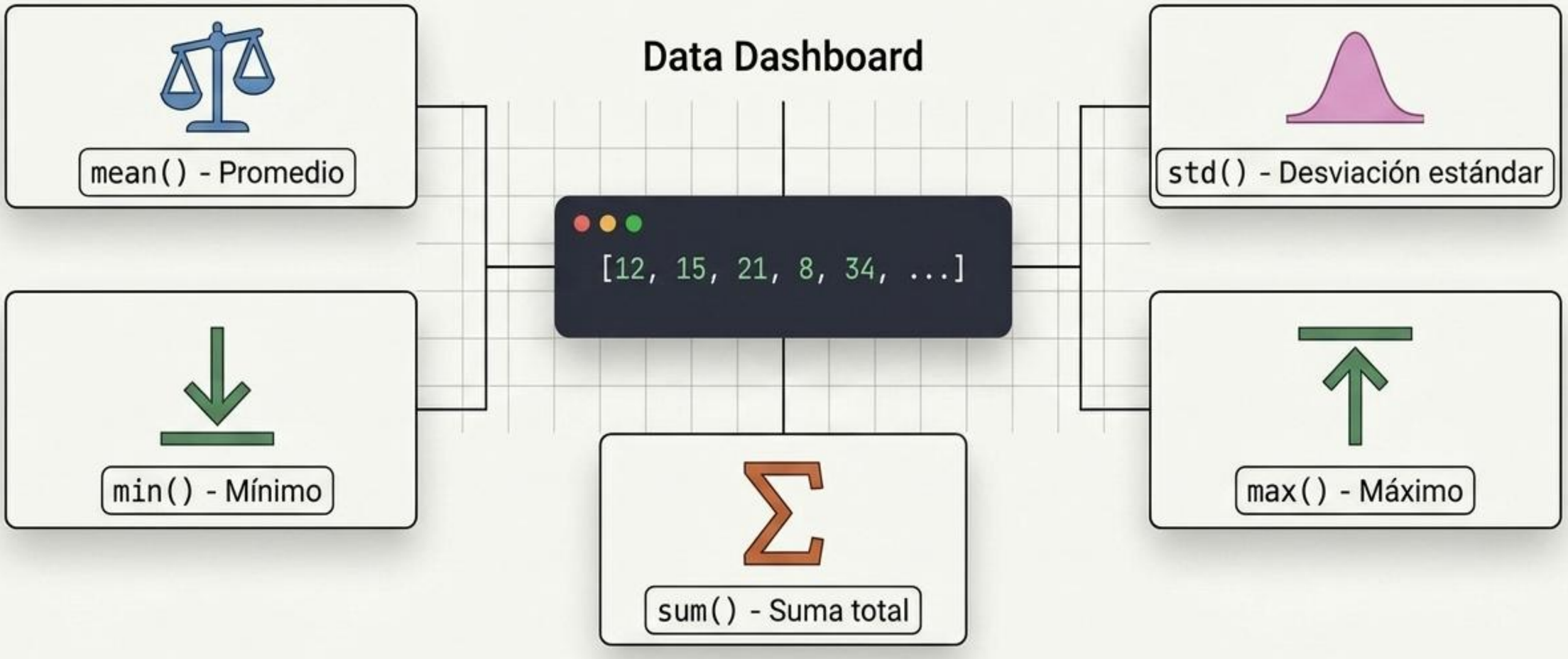
Implementación en C (Instantáneo)

```
array1 = np.array([1, 2, 3, 4, 5])
resultado = array1 + 10    # Operación en bloque
suma = array1 + array1     # Elemento a elemento
```

Recomendación: ¡Cronometra esto en laboratorio con 1 millón de elementos!

Estadística Descriptiva en una Sola Línea

Cálculos matemáticos aplicados al instante sobre el arreglo completo.
Sin necesidad de iterar manualmente.



Redes Frágiles: El Timeout No Es Opcional

Anatomy of a Robust Request

```python
import requests

try:
    respuesta = requests.get(url, params=parametros, timeout=5)
    respuesta.raise_for_status()
except requests.exceptions.RequestException as error:
    print(f'Error crítico: {error}')
```

Previene bloqueos infinitos si el servidor falla (Oxylabs, 2026).

Captura errores HTTP automáticamente (ej. 404, 500).

La clase padre que agrupa `ConnectionError` y `HTTPError`.

Extrayendo Datos Reales: API Open-Meteo (Huancayo)

✅ API pública sin clave de autenticación (API Key).
✅ Datos meteorológicos históricos y pronósticos.

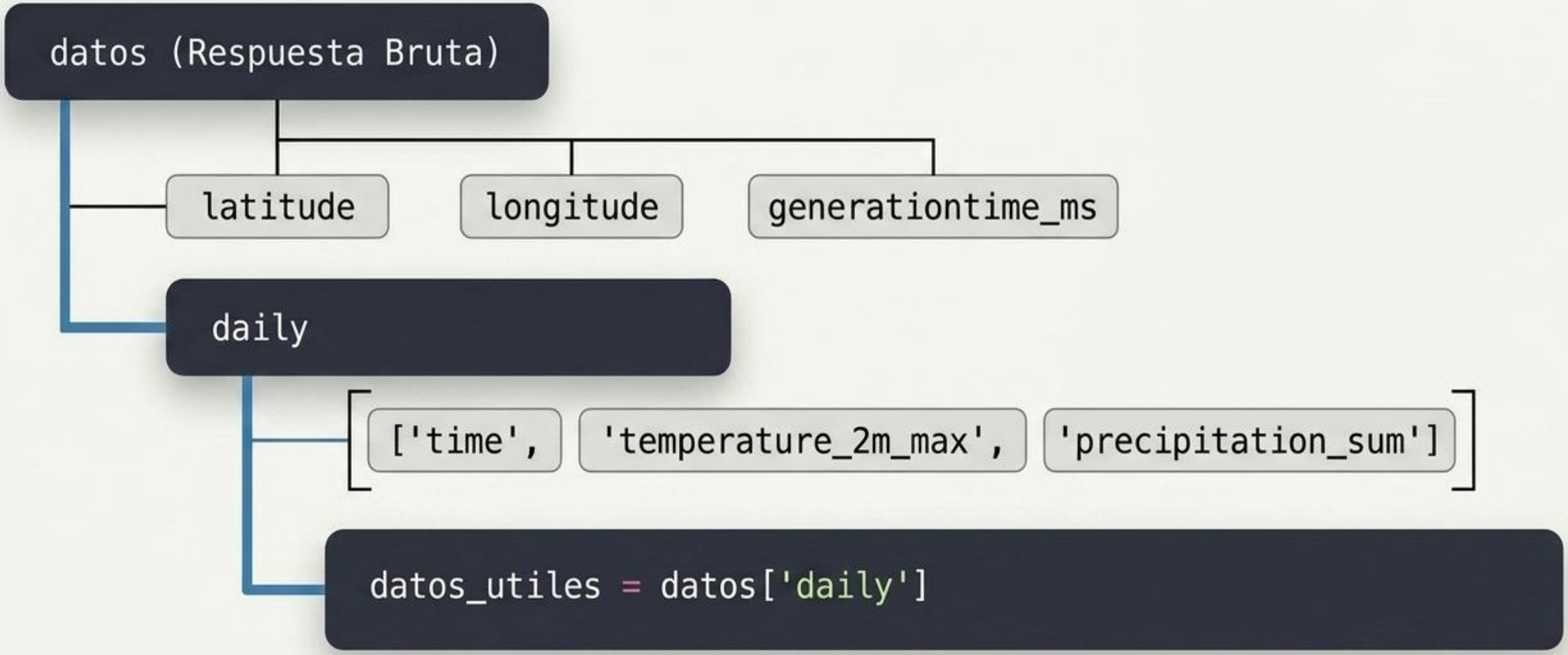


```
parametros = {
    'latitude': -12.07,   # Huancayo
    'longitude': -75.21,
    'daily':
'temperature_2m_max,precipitation_sum',
    'timezone': 'America/Lima'
}
respuesta = requests.get(url,
params=parametros, timeout=5)

datos = respuesta.json()
```

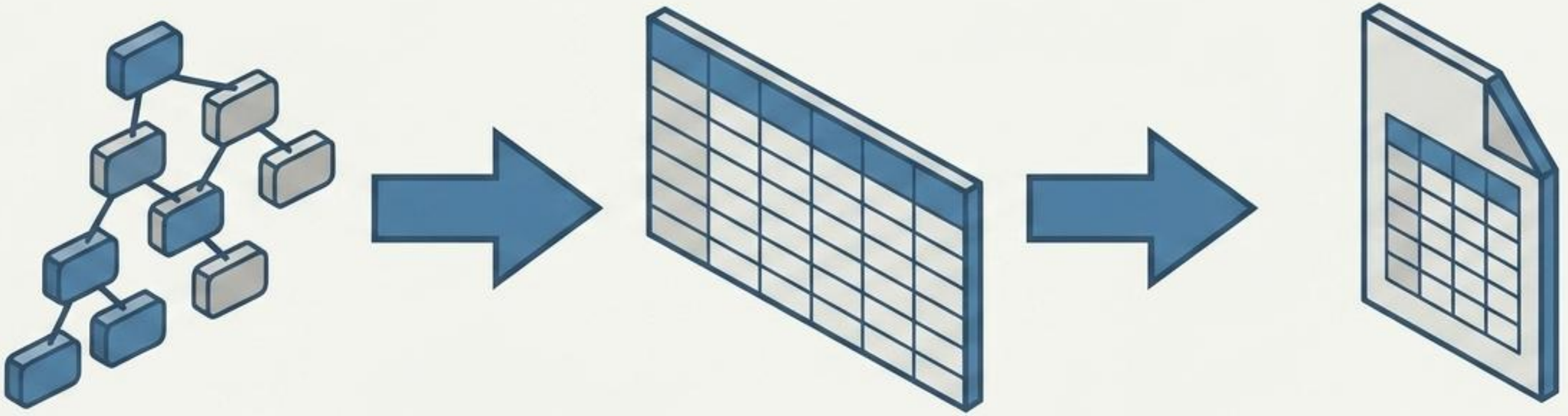
El Laberinto JSON: Navegando Datos Anidados

JSON representa datos jerárquicos (diccionarios dentro de listas). Nuestro objetivo es extraer únicamente la llave 'daily'.



De JSON a CSV: Aplanando la Estructura de Datos

Las APIs hablan en jerarquías (JSON). El análisis de datos requiere cuadrículas (DataFrame). Pandas es el traductor.



Origen Web

`JSON Anidado (datos['daily'])`

Transformación en RAM

```
pd.DataFrame()
```

Almacenamiento Local

```
df.to_csv()
```

Buenas Prácticas CSV: Evitando Desastres Silenciosos

Un CSV es solo texto plano. Las comas, tabulaciones y codificaciones varían según la región.

Problema Común ⚠️	Solución en Pandas ✅
Todo el texto se agrupa en una sola columna.	`sep=';'` (Ajustar delimitador)
Caracteres rotos en acentos o eñes (ñ).	`encoding='utf-8'` (O usar chardet)
El archivo tiene líneas corruptas que frenan la lectura.	`on_bad_lines='skip'` (Ignorar errores)

Pandas: Cargando y Explorando el DataFrame

El primer paso tras cargar los datos es entender su anatomía estructural.

```
> 1  df = pd.DataFrame(datos['daily'])
  2  df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Data columns (total 3 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   time             7 non-null      object
 1   temperature_max  7 non-null      float64
 2   precipitation    7 non-null      float64
```

Transformación de Columnas: Limpieza Básica

BEFORE

df['time']
'2023-10-01'
'2023-10-02'
'2023-10-03'
'2023-10-04'

Tipo: object (Texto estático)



AFTER

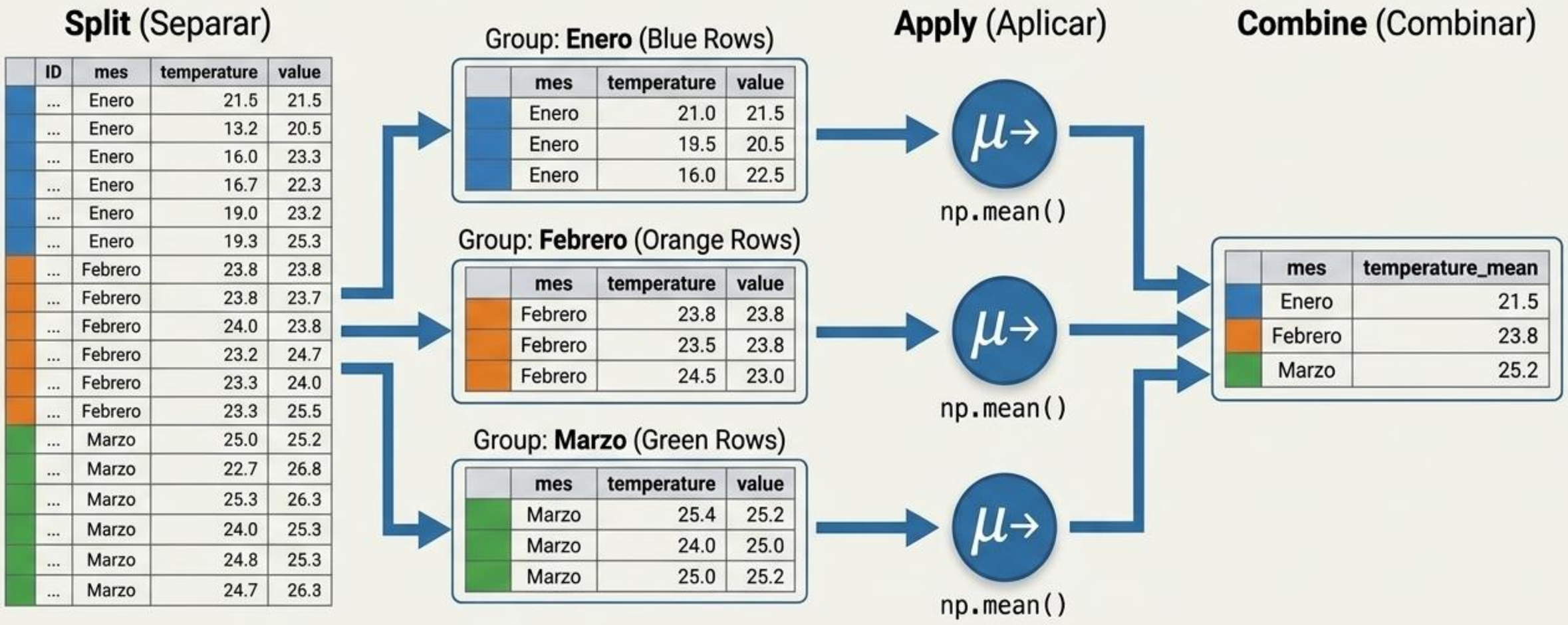
df['time']
2023-10-01
2023-10-02
2023-10-03
2023-10-04

Tipo: datetime64 (Lista para análisis temporal)

```
df['time'] = pd.to_datetime(df['time'])
```

Agrupaciones y Agregaciones (El Modelo Groupby)

La filosofía Split-Apply-Combine permite resumir millones de filas en segundos.



	ID	mes	temperature	value
	...	Enero	21.5	21.5
	...	Enero	13.2	20.5
	...	Enero	16.0	23.3
	...	Enero	16.7	22.3
	...	Enero	19.0	23.2
	...	Enero	19.3	25.3
	...	Febrero	23.8	23.8
	...	Febrero	23.8	23.7
	...	Febrero	24.0	23.8
	...	Febrero	23.2	24.7
	...	Febrero	23.3	24.0
	...	Febrero	23.3	25.5
	...	Marzo	25.0	25.2
	...	Marzo	22.7	26.8
	...	Marzo	25.3	26.3
	...	Marzo	24.0	25.3
	...	Marzo	24.8	25.3
	...	Marzo	24.7	26.3

	mes	temperature	value
	Enero	21.0	21.5
	Enero	19.5	20.5
	Enero	16.0	22.5

	mes	temperature	value
	Febrero	23.8	23.8
	Febrero	23.5	23.8
	Febrero	24.5	23.0

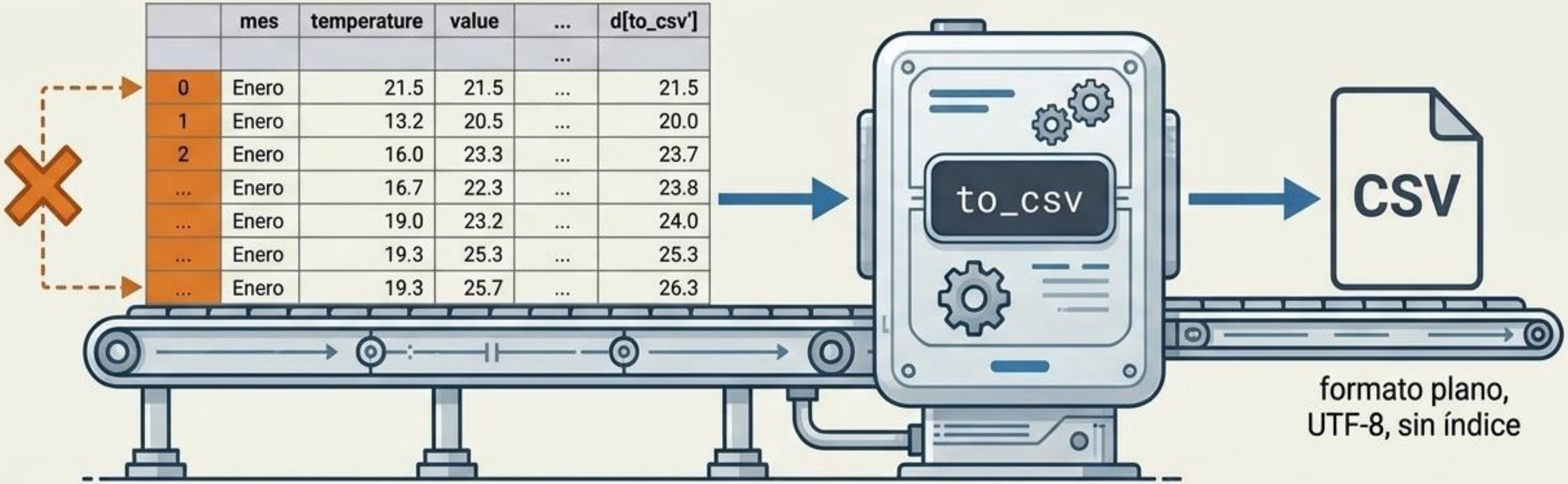
	mes	temperature	value
	Marzo	25.4	25.2
	Marzo	24.0	25.0
	Marzo	25.0	25.2

	mes	temperature_mean
	Enero	21.5
	Febrero	23.8
	Marzo	25.2

```
df.groupby('mes')['temperature'].mean()
```

Exportación de Resultados: Cerrando el Ciclo

El pipeline culmina exportando el valor generado, listo para consumo en Excel, PowerBI o modelos de Machine Learning.




```
# Guardar asegurando trazabilidad y formato limpio
df.to_csv('pronostico_huancayo.csv', index=False, encoding='utf-8')
```

Actividad Práctica:
Checklist de Laboratorio

- [ ]  Paso 1: Ejecutar petición a Open-Meteo (Huancayo) garantizando un `timeout=5`.
- [ ]  Paso 2: Navegar el JSON resultante y aislar el nodo `daily`.
- [ ]  Paso 3: Convertir a DataFrame y transformar la columna temporal con `pd.to_datetime()`.
- [ ]  Paso 4: Exportar a CSV validando delimitadores y descartando el índice.

Referencias Bibliográficas

Codiga (2026). Python Best Practices: always use a timeout with requests.

Kanaries (2026). Pandas read_csv: The Definitive Guide.

McKinney, W. (2022). Python for Data Analysis, 3.ª ed. (Capítulo 4: NumPy Basics).

Open-Meteo (2026). Free Open-Source Weather API.

pandas / NumPy documentation (2026). Official API references.